

NumPy II — Linear Algebra Intuition & Descriptive Statistics

By the end of today you will understand vectors, matrices, and dot products — the actual math that powers every ML model — without any scary formulas or proofs. You will also master descriptive statistics with NumPy and be able to summarise any dataset confidently. This is the last pure-math day before we move into Pandas and real data.

Mac users reminder: Activate your virtual environment first before running any code. [source](#)
`venv/bin/activate` Then run files using `python3 filename.py`

SECTION 01 Why Linear Algebra? What Does It Have to Do With ML?

The phrase 'linear algebra' makes most people nervous. It sounds like advanced university mathematics. But here is the truth: you use it every time you open a spreadsheet, look at a table of data, or calculate an average. The concepts are simple. The words just sound scary.

Linear algebra is just organised math on collections of numbers.

A single number (like your age: 22) is a scalar.

A row of numbers (like your exam marks: [85, 92, 78, 95]) is a vector.

A table of numbers (like all students' marks across all subjects) is a matrix.

Every single ML algorithm — from the simplest linear regression to the most complex neural network — is built on operations involving scalars, vectors, and matrices. Learn this and you will understand what is happening inside every model you build.

SECTION 02 Scalars, Vectors and Matrices — The Building Blocks

Scalar — a single number

A scalar is just one number. No direction, no collection. Just a value. Temperature: 36.6°. Price: Rs. 299. Your age: 22. All scalars.

```
import numpy as np
```

```

temperature = 36.6      # A plain Python float – a scalar
learning_rate = 0.01    # A hyperparameter in ML – also a scalar

# As a NumPy scalar
s = np.float64(36.6)
print(s)                # 36.6
print(s.ndim)           # 0 – zero dimensions, it is a scalar

```

Output:

```

36.6
0

```

Vector — a row of numbers

Vector

What it is: An ordered list of numbers. In ML, one row in your dataset is a vector. Each number in the vector is called a component or feature.

Real-life analogy: Your student report card is a vector. It contains your marks in multiple subjects: [85, 92, 78, 95]. Each subject's mark is one component of that vector. The entire row describes YOU as a data point.

In Python: `student = np.array([85, 92, 78, 95])` # A 1D vector with 4 components

```

import numpy as np

# A student's features: age, study_hours, attendance%, prev_score
student_features = np.array([20, 6.5, 92.0, 78])

print(student_features)      # [20.   6.5  92.   78. ]
print("Shape:", student_features.shape) # (4,) – 4 features
print("Length:", len(student_features))  # 4

# In ML this single row is called a 'feature vector'
# Each column (age, hours, attendance, score) is a 'feature'

```

Output:

```

[20.   6.5  92.   78. ]
Shape: (4,)
Length: 4

```

In machine learning, EVERY data point is a vector.

A house for sale: [1200, 3, 2, 15] — [sqft, bedrooms, bathrooms, age]

An email for spam: [1, 0, 1, 0, 1] — [has_link, urgent, free_offer, ...]

A customer: [35, 72000, 3, 1] — [age, income, purchases, churned]

The model learns from these vectors to make predictions.

Matrix — a table of numbers

Matrix

What it is: A 2D array of numbers arranged in rows and columns. In ML, the entire dataset is a matrix. Each row is one data point (one person, one transaction, one image). Each column is one feature.

Real-life analogy: Your entire class's marks table is a matrix. Rows = students. Columns = subjects. Every number has a clear address: row 2, column 3 = Arjun's English marks.

In Python: `marks = np.array([[85, 90, 78], [72, 88, 95], [91, 76, 83]])` # 3x3 matrix

```
import numpy as np

# Dataset: 5 customers, 3 features each
# Columns: [age, annual_income_thousands, spending_score]
customers = np.array([
    [25, 35, 81],
    [47, 82, 44],
    [31, 28, 55],
    [22, 16, 20],
    [35, 60, 76]
])

print(customers.shape) # (5, 3) — 5 rows (customers), 3 cols (features)
print(customers.ndim) # 2 — it is a 2D matrix

# In ML we call this the feature matrix, often written as X
X = customers
print(f"Dataset has {X.shape[0]} samples and {X.shape[1]} features")
```

Output:

```
(5, 3)
2
Dataset has 5 samples and 3 features
```

The standard ML naming convention you will see everywhere:

X — The feature matrix. Shape: (n_samples, n_features)

y — The target vector. Shape: (n_samples,)

If you have 1000 customers, 5 features each, and a churn label:

X has shape (1000, 5) — 1000 rows, 5 columns
y has shape (1000,) — 1000 labels, one per customer

SECTION 03 Dot Product — The Most Important Operation in ML

If there is one single mathematical operation you must understand before starting machine learning, it is the dot product. Every prediction in linear regression, every neuron in a neural network, every similarity calculation uses dot products underneath.

What is a dot product?

A dot product takes two vectors of the same length, multiplies each pair of matching elements, and adds all those products together into one single number.

```
Vector A:  [2,  3,  4]
Vector B:  [1,  5,  2]
```

```
Step 1 — Multiply matching pairs:
      2*1   3*5   4*2
      = 2 + 15 + 8
```

```
Step 2 — Add all products:
      = 25
```

```
Dot product of A and B = 25
```

```
import numpy as np

a = np.array([2, 3, 4])
b = np.array([1, 5, 2])

# Method 1: np.dot()
result = np.dot(a, b)
print(result)      # 25

# Method 2: @ operator (cleaner, preferred in modern Python)
result2 = a @ b
print(result2)     # 25

# Manual verification
manual = (2*1) + (3*5) + (4*2)
print(manual)      # 25
```

Output:

25
25
25

Why dot product matters in ML — a real example

Here is the exact situation you will face in machine learning. A linear regression model makes predictions using weights. The prediction is simply the dot product of the feature vector and the weight vector.

Predicting a house price:

Features of the house:

size=1500 sqft, bedrooms=3, age=10 years, distance_from_city=5 km

Feature vector $x = [1500, 3, 10, 5]$

The model has learned weights (importance of each feature):

Weights $w = [200, 50000, -2000, -10000]$

(size adds Rs.200/sqft, each bedroom adds 50k, older = cheaper, far = cheaper)

Predicted price = dot product of x and w

$= (1500 \times 200) + (3 \times 50000) + (10 \times -2000) + (5 \times -10000)$

$= 300000 + 150000 - 20000 - 50000$

$= \text{Rs. } 3,80,000$

```
import numpy as np

# House features: [size_sqft, bedrooms, age_years, distance_km]
house = np.array([1500, 3, 10, 5])

# Model weights (what the model learned from training data)
weights = np.array([200, 50000, -2000, -10000])

# Prediction = dot product
predicted_price = np.dot(house, weights)
print(f"Predicted house price: Rs. {predicted_price:,}")

# This is LITERALLY what linear regression does
# Just with thousands of houses and the weights found automatically
```

Output:

Predicted house price: Rs. 3,80,000

You just computed a linear regression prediction by hand using a dot product. When you use scikit-learn's LinearRegression in Phase 2, this is exactly what is happening inside. The algorithm just finds the best weights automatically from the data.

Matrix multiplication — many dot products at once

Matrix multiplication extends the dot product to work on entire matrices at once. When you multiply two matrices, you are computing the dot product between every row of the first matrix and every column of the second matrix simultaneously.

Rule for matrix multiplication:

Matrix A shape: (m, n) x Matrix B shape: (n, p) = Result shape: (m, p)

The INNER dimensions must match (both n).

The result has the OUTER dimensions (m rows, p columns).

Example: (3, 4) x (4, 2) = (3, 2) ✓ Works

(3, 4) x (3, 2) = ERROR ✗ Inner dims 4 and 3 do not match

```
import numpy as np

# 3 students, 4 subject marks each — shape (3, 4)
marks = np.array([
    [85, 90, 78, 92],
    [72, 88, 95, 80],
    [91, 76, 83, 87]
])

# Credit weights per subject (how much each counts) — shape (4, 1)
# Maths=40%, Science=30%, English=20%, Hindi=10%
weights = np.array([[0.40], [0.30], [0.20], [0.10]])

# Weighted average for each student = matrix multiplication
# Shape: (3,4) @ (4,1) = (3,1)
weighted_avg = marks @ weights
print("Weighted averages:")
print(weighted_avg)

students = ["Rahul", "Priya", "Arjun"]
for i, name in enumerate(students):
    print(f" {name}: {weighted_avg[i,0]:.1f}")
```

Output:

```
Weighted averages:
[[85.6]
 [82.7]
 [84.7]]
Rahul: 85.6
```

```
Priya: 82.7
Arjun: 84.7
```

Transpose — flipping rows and columns

Transposing a matrix swaps its rows and columns. A matrix of shape (3, 4) becomes shape (4, 3) after transposing. This comes up constantly in ML when shapes do not align for multiplication.

```
import numpy as np

A = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
print("Original shape:", A.shape)    # (2, 3)
print(A)

# Transpose using .T
A_T = A.T
print("Transposed shape:", A_T.shape) # (3, 2)
print(A_T)
```

Output:

```
Original shape: (2, 3)
[[1 2 3]
 [4 5 6]]
Transposed shape: (3, 2)
[[1 4]
 [2 5]
 [3 6]]
```

SECTION 04 The Only Linear Algebra You Need For ML — Quick Reference

You do not need to memorise proofs or derive formulas. You need to understand what each operation does and when to use it. This table covers everything you will encounter in this program.

Operation	NumPy code	When you will use it in ML
Dot product	<code>np.dot(a,b)</code> or <code>a @ b</code>	Every prediction in linear models. Neuron output in neural networks.
Matrix multiply	<code>A @ B</code>	Passing data through neural network layers.
Transpose	<code>A.T</code>	Fixing shape mismatches. Computing covariance matrices.

Vector norm	<code>np.linalg.norm(v)</code>	Measuring distance between two data points.
Matrix inverse	<code>np.linalg.inv(A)</code>	Solving linear equations. Used in some regression formulas.
Eigenvalues	<code>np.linalg.eig(A)</code>	Principal Component Analysis (PCA) in Phase 2.
Element-wise multiply	<code>a * b</code>	Applying masks. Gradient calculations.

```
import numpy as np

# Vector norm — the 'length' of a vector
# Used to measure how similar or different two data points are
point_a = np.array([3, 4])
norm = np.linalg.norm(point_a)
print(f"Norm of [3,4]: {norm}")          # 5.0 (Pythagoras: sqrt(3^2 + 4^2))

# Distance between two points (used in KNN algorithm, Day 20)
point_b = np.array([0, 0])
distance = np.linalg.norm(point_a - point_b)
print(f"Distance: {distance}")          # 5.0
```

Output:

```
Norm of [3,4]: 5.0
Distance: 5.0
```

SECTION 05 Descriptive Statistics With NumPy

Before building any ML model, you must understand your data. Descriptive statistics are the numbers that summarise and describe a dataset. This is the first thing every data scientist does when they receive new data.

If you skip understanding your data and go straight to building models, you will get wrong answers that look correct. Always explore and summarise your data first. Always.

Measures of Central Tendency — where is the data centred?

These three measures tell you what a 'typical' value in your dataset looks like. They each give a different perspective and can differ dramatically from each other.

Mean — the average

Add all values and divide by the count. The mean is the most used but the most sensitive to extreme values (outliers).

```
import numpy as np

salaries = np.array([30000, 35000, 32000, 28000, 31000, 500000])
# Note: one person earns 5 lakhs – an outlier

mean = np.mean(salaries)
print(f"Mean salary: Rs. {mean:,.0f}")
# Output: Rs. 1,09,333 – MISLEADING because of the outlier!
```

Output:

Mean salary: Rs. 1,09,333

The mean is pulled heavily by outliers. One salary of 5 lakhs dragged the mean from around 31,000 to over 1 lakh. Most people in this team earn around 31,000 but the mean says 1.09 lakh. This is why you never use just one statistic.

Median — the middle value

Sort all values and pick the middle one. The median is resistant to outliers because it only cares about position, not the actual value of extreme numbers.

```
import numpy as np

salaries = np.array([30000, 35000, 32000, 28000, 31000, 500000])

median = np.median(salaries)
print(f"Median salary: Rs. {median:,.0f}")
# Sorted: [28000, 30000, 31000, 32000, 35000, 500000]
# Middle two: 31000 and 32000, average = 31500
# Output: Rs. 31,500 – much more representative!
```

Output:

Median salary: Rs. 31,500

The median of Rs. 31,500 tells a much more honest story about what most people earn. This is why newspapers report 'median household income' not 'mean household income'. Use median when you suspect outliers in your data.

Mode — the most frequent value

The value that appears most often. NumPy does not have a built-in mode function, but we can find it. Most useful for categorical data.

```
import numpy as np
```

```

grades = np.array(["A", "B", "A", "C", "A", "B", "A", "D", "B"])

# Find unique values and their counts
values, counts = np.unique(grades, return_counts=True)
mode_index      = np.argmax(counts)
mode            = values[mode_index]

print(f"Grades:  {values}")
print(f"Counts:  {counts}")
print(f"Mode (most common grade): {mode}")

```

Output:

```

Grades:  ['A' 'B' 'C' 'D']
Counts:  [4 3 1 1]
Mode (most common grade): A

```

Mean vs Median vs Mode — when to use which

Statistic	Use when...	Watch out for...
Mean	Data is fairly symmetric with no extreme outliers. Exam scores, temperatures.	Pulled heavily by outliers. One Rs.5 lakh salary ruins the whole picture.
Median	Data has outliers or is skewed. Salaries, house prices, income data.	Does not use all values, so misses information about extremes.
Mode	Categorical data or discrete values. Most common product, most common grade.	Might not exist (all values unique) or there can be multiple modes.

Measures of Spread — how scattered is the data?

Central tendency tells you where data is centred. Spread tells you how wide the data is around that centre. Two datasets can have the same mean but completely different spreads.

Variance and Standard Deviation

Variance measures how far each value is from the mean, on average. Standard deviation is just the square root of variance, in the same units as the original data, which makes it easier to interpret.

```

import numpy as np

# Two classes with the same average but different spreads
class_a = np.array([70, 72, 74, 75, 73, 71, 76, 74]) # Consistent class
class_b = np.array([45, 92, 38, 99, 55, 88, 30, 98]) # Wildly varied
class

```

```
print(f"Class A mean: {np.mean(class_a):.1f}")
print(f"Class B mean: {np.mean(class_b):.1f}")
print("")
print(f"Class A std deviation: {np.std(class_a):.1f}")
print(f"Class B std deviation: {np.std(class_b):.1f}")
print("")
print(f"Class A variance: {np.var(class_a):.1f}")
print(f"Class B variance: {np.var(class_b):.1f}")
```

Output:

```
Class A mean: 73.1
Class B mean: 68.1
```

```
Class A std deviation: 1.9
Class B std deviation: 26.8
```

```
Class A variance: 3.6
Class B variance: 718.1
```

Both classes have similar averages (73 and 68) but completely different stories.

Class A std = 1.9 — Everyone scored between roughly 71 and 75. Very consistent.

Class B std = 26.8 — Scores range from 30 to 99. Wildly inconsistent.

In ML: high variance in a feature means it contains a lot of information.

Low variance means the feature is nearly constant and probably useless for prediction.

Percentiles and Quartiles

The Nth percentile is the value below which N% of the data falls. Percentiles are used to understand the distribution of data and to detect outliers.

```
import numpy as np

scores = np.array([45, 52, 58, 61, 64, 67, 70, 73, 76, 80,
                  82, 85, 88, 91, 93, 95, 97, 98, 99, 100])

# Key percentiles
p25 = np.percentile(scores, 25)    # Q1 - 25th percentile
p50 = np.percentile(scores, 50)    # Q2 - Median (same as np.median)
p75 = np.percentile(scores, 75)    # Q3 - 75th percentile

print(f"25th percentile (Q1): {p25}")
print(f"50th percentile (Q2, Median): {p50}")
print(f"75th percentile (Q3): {p75}")
```

```
# IQR – Interquartile Range – the middle 50% spread
iqr = p75 - p25
print(f"IQR: {iqr}")

# Outlier detection using IQR method
lower_fence = p25 - 1.5 * iqr
upper_fence = p75 + 1.5 * iqr
print(f"Outlier bounds: [{lower_fence:.1f}, {upper_fence:.1f}]")

outliers = scores[(scores < lower_fence) | (scores > upper_fence)]
print(f"Outliers found: {outliers}")
```

Output:

```
25th percentile (Q1): 64.75
50th percentile (Q2, Median): 81.0
75th percentile (Q3): 93.5
IQR: 28.75
Outlier bounds: [21.6, 136.6]
Outliers found: []
```

The IQR method for detecting outliers is the most common approach in data science.

Lower fence = $Q1 - 1.5 * IQR$

Upper fence = $Q3 + 1.5 * IQR$

Any value outside these fences is considered an outlier.

This is exactly what the box plot visualises — which you will see in Matplotlib on Day 9.

SECTION 06 Correlation — Do Two Things Move Together?

Correlation

What it is: A number between -1 and +1 that measures how strongly two variables move together. +1 means they move perfectly together. -1 means they move perfectly opposite. 0 means no relationship.

Real-life analogy: Think of study hours and exam scores. Students who study more tend to score higher. That is a positive correlation. Or think of absences and scores: more absences = lower scores. That is a negative correlation.

In Python: `np.corrcoef(hours studied, exam scores)`

```
import numpy as np

# 8 students: hours studied and exam score
hours_studied = np.array([1, 2, 3, 4, 5, 6, 7, 8])
```

```

exam_scores = np.array([45, 52, 58, 65, 72, 78, 85, 92])
absences    = np.array([8, 7, 6, 5, 4, 3, 2, 1])

# Correlation matrix between hours and scores
corr1 = np.corrcoef(hours_studied, exam_scores)
print("Hours vs Scores correlation:")
print(corr1)
print(f"Correlation value: {corr1[0,1]:.3f}")

# Correlation between absences and scores
corr2 = np.corrcoef(absences, exam_scores)
print(f"\nAbsences vs Scores correlation: {corr2[0,1]:.3f}")

```

Output:

```

Hours vs Scores correlation:
[[1.    0.999]
 [0.999 1.    ]]
Correlation value: 0.999

Absences vs Scores correlation: -0.999

```

Reading the correlation output:

`np.corrcoef` returns a matrix. The value at [0,1] (or [1,0]) is your correlation. The diagonal (1.0) is each variable correlated with itself — always 1.

0.999 → Almost perfect positive correlation. More hours = higher score.
 -0.999 → Almost perfect negative correlation. More absences = lower score.

In ML, you will use correlation to find which features are related to the target and to spot features that are too similar to each other (multicollinearity).

Correlation	Strength	Example
0.9 to 1.0	Very strong positive	Study hours vs exam score
0.7 to 0.9	Strong positive	Exercise vs fitness level
0.4 to 0.7	Moderate positive	Income vs spending
0.0 to 0.4	Weak or no relationship	Shoe size vs exam score
-0.4 to 0.0	Weak negative	Sleep hours vs stress
-0.7 to -0.4	Moderate negative	Absences vs grades
-1.0 to -0.7	Strong negative	Temperature vs heater usage

This is the kind of analysis you will do at the very beginning of every ML project. Load the data, understand its shape, check for issues, compute statistics, look for relationships. This is called Exploratory Data Analysis (EDA) and Phase 1 of the program builds up to you doing this confidently.

```
import numpy as np

# — Simulated dataset: 10 students —————
# Columns: study_hours, attendance_pct, prev_score, exam_score
np.random.seed(42)

study_hours = np.array([2, 5, 1, 7, 4, 6, 3, 8, 2, 5])
attendance = np.array([60, 85, 45, 95, 78, 90, 65, 98, 55, 82])
prev_score = np.array([50, 72, 38, 88, 65, 80, 55, 91, 42, 70])
exam_score = np.array([48, 75, 35, 92, 68, 83, 52, 96, 40, 74])

# Stack into a feature matrix X
X = np.column_stack([study_hours, attendance, prev_score])
y = exam_score

print("==== DATASET OVERVIEW ====")
print(f"Samples : {X.shape[0]}")
print(f"Features: {X.shape[1]}")
print(f"Target : exam_score (y)")

print("\n==== DESCRIPTIVE STATISTICS ====")
feature_names = ["study_hours", "attendance%", "prev_score"]
for i, name in enumerate(feature_names):
    col = X[:, i]
    print(f"\n{name}:")
    print(f" Mean : {np.mean(col):.2f}")
    print(f" Median : {np.median(col):.2f}")
    print(f" Std : {np.std(col):.2f}")
    print(f" Min : {np.min(col):.2f}")
    print(f" Max : {np.max(col):.2f}")

print("\n==== CORRELATIONS WITH EXAM SCORE ====")
for i, name in enumerate(feature_names):
    corr = np.corrcoef(X[:, i], y)[0, 1]
    print(f" {name:15}: {corr:.3f}")

print("\n==== OUTLIER CHECK (IQR METHOD) ====")
q1, q3 = np.percentile(y, [25, 75])
iqr = q3 - q1
low = q1 - 1.5 * iqr
high = q3 + 1.5 * iqr
outlier_mask = (y < low) | (y > high)
print(f" Outliers in exam_score: {np.sum(outlier_mask)}")

print("\n==== PASS / FAIL SUMMARY ====")
```

```
passed = np.sum(y >= 40)
failed = np.sum(y < 40)
print(f" Passed: {passed} / {len(y)}")
print(f" Failed: {failed} / {len(y)}")
print(f" Pass rate: {passed/len(y)*100:.0f}%")
```

Output:

```
===== DATASET OVERVIEW =====
Samples : 10
Features: 3
Target  : exam_score (y)

===== DESCRIPTIVE STATISTICS =====

study_hours:
  Mean    : 4.30
  Median  : 4.50
  Std     : 2.21
  Min     : 1.00
  Max     : 8.00

===== CORRELATIONS WITH EXAM SCORE =====
  study_hours    : 0.997
  attendance%    : 0.994
  prev_score     : 0.998

===== OUTLIER CHECK (IQR METHOD) =====
  Outliers in exam_score: 0

===== PASS / FAIL SUMMARY =====
  Passed: 9 / 10
  Failed: 1 / 10
  Pass rate: 90%
```

This output tells you: all three features are very strongly correlated with exam score (all above 0.99), there are no outliers, and 90% of students passed. In a real project with thousands of rows, this analysis would take 30 seconds and immediately tell you which features matter most.

SECTION 08 Interview Prep — Questions to Answer After Today

These questions bridge mathematics and machine learning. Being able to answer them shows you understand WHY the math matters, not just how to compute it.

Q — What is a dot product and why is it important in machine learning?

A dot product takes two vectors of the same length, multiplies their corresponding elements, and sums the results into a single number. In machine learning, dot products are fundamental

because every linear model prediction is a dot product: the model multiplies each feature value by its learned weight and sums the results to get the prediction. In neural networks, each neuron computes a dot product of its inputs and weights, then applies an activation function. Basically, any time you see a weighted sum of features, that is a dot product.

Q — What is the difference between mean and median? When would you use each?

The mean is the arithmetic average — sum all values and divide by count. The median is the middle value when data is sorted. The key difference is sensitivity to outliers. The mean is heavily influenced by extreme values: one billionaire in a room raises the mean income dramatically but does not change the median much. Use the mean when data is roughly symmetric with no extreme outliers, like exam scores. Use the median when data is skewed or has outliers, like salaries, house prices, or income data. In data science, always report both and compare them.

Q — What does standard deviation tell you about a dataset?

Standard deviation measures how spread out the values are around the mean. A low standard deviation means values are clustered tightly around the mean — the data is consistent. A high standard deviation means values are spread far from the mean — the data is varied. In machine learning, standard deviation is used in feature scaling (StandardScaler divides by std to normalise features), in detecting outliers (values more than 3 std from mean are often outliers), and in understanding which features have meaningful variation versus those that are nearly constant.

Q — What is correlation and how do you interpret a correlation of -0.85?

Correlation is a number between -1 and +1 that measures the strength and direction of the linear relationship between two variables. A correlation of -0.85 indicates a strong negative relationship: as one variable increases, the other tends to decrease, and this relationship is fairly consistent (85% of the way to a perfect linear negative relationship). In a practical example, the correlation between student absences and exam scores might be -0.85, meaning students who miss more classes tend to score significantly lower. In feature selection for ML, we look for features with high absolute correlation to the target and remove features that are highly correlated with each other.

SECTION 09 Words to Know

Term	Definition
Scalar	A single number with no direction or dimension. Example: temperature = 36.6
Vector	An ordered list of numbers. One row in a dataset. A data point's features.
Matrix	A 2D table of numbers. The entire dataset. Rows=samples, Cols=features.
Feature vector	A vector representing one data point's input values. Called x in ML.
Feature matrix	The full dataset matrix of all samples. Called X in ML.
Dot product	Multiply matching pairs of two vectors and sum the results. Core of all predictions.

Matrix multiply	Computing dot products between all rows and columns of two matrices simultaneously.
Transpose	Flipping rows and columns of a matrix. A.T in NumPy.
Norm	The 'length' of a vector. Used to measure distance between points.
Mean	Sum of all values divided by count. Sensitive to outliers.
Median	The middle value when sorted. Resistant to outliers.
Mode	The most frequently occurring value in a dataset.
Variance	Average of squared distances from the mean. Measures spread.
Standard deviation	Square root of variance. Same units as data. Easier to interpret.
Percentile	The value below which N% of data falls. 50th percentile = median.
IQR	Interquartile Range. Q3 - Q1. Spread of the middle 50% of data.
Correlation	Strength of linear relationship between two variables. -1 to +1.
Outlier	A data point that is very different from the rest. Can distort analysis.
EDA	Exploratory Data Analysis. The first step of any data project.

SECTION HW Homework — Submit Before Day 7

Create `day6_homework.py`, activate your venv first, and push to GitHub before midnight. This is your last NumPy homework before Pandas starts on Day 7.

Exercise 1 — Dot Product Prediction (Beginner)

You are given the features of three houses and a set of model weights. Use `np.dot()` to predict the price of each house. Features: `[size_sqft, bedrooms, bathrooms, age_years]`. Weights: `[150, 45000, 30000, -1500]`. Create the feature matrix and compute all three predictions at once using matrix multiplication.

House 1: `[1200, 2, 1, 20]`
House 2: `[1800, 3, 2, 5]`
House 3: `[2500, 4, 3, 2]`

Exercise 2 — Statistics Summary Function (Beginner-Intermediate)

Write a function called `describe(arr, name)` that takes a NumPy array and prints a complete statistical summary: count, mean, median, standard deviation, min, 25th percentile, 75th percentile, and max. Test it on three different arrays: exam scores, daily temperatures, and product prices of your choice.

Exercise 3 — Outlier Detector (Intermediate)

Write a function called `find_outliers(arr)` that uses the IQR method to find and return all outlier values in a NumPy array. Test it on this array of employee salaries: `[28000, 31000, 29500, 32000, 30000, 28500, 550000, 29000, 31500, 27000, 480000]`. Print the outliers and state how many there are.

Exercise 4 — Correlation Analysis (Intermediate)

Create two NumPy arrays of 10 values each: temperature (ranging from 20 to 42 degrees) and ice cream sales (generally higher on hotter days, add some randomness). Create another array: hot chocolate sales (generally lower on hotter days). Calculate and print the correlation of temperature with both products. Confirm that ice cream has positive correlation and hot chocolate has negative correlation.

Exercise 5 — Mini NumPy EDA (Intermediate)

Create a dataset of 15 students with three features: `hours_slept` (5 to 9), `stress_level` (1 to 10), and `exam_score` (40 to 100). Use realistic relationships — more sleep and less stress should loosely correspond to higher scores. Then: print shape, compute full descriptive stats for each feature, compute correlation of each feature with `exam_score`, check for outliers in `exam_score`, and print the percentage of students who scored above 75.

Stretch Challenge — Tier A students

Implement the following from scratch using only NumPy (no scipy or sklearn): (1) A function that computes the covariance between two arrays without using `np.cov()`. Covariance formula: $\text{cov}(x,y) = \text{mean}((x - \text{mean}(x)) * (y - \text{mean}(y)))$. (2) A function that computes the Pearson correlation coefficient from scratch using your covariance function. Verify your results match `np.corrcoef()` on the same data.